

A Discipline of Partial Audit Classification

On the Correct Treatment of Undefined Audit Assertions

Duston Moore

the date of receipt and acceptance should be inserted later

Abstract Audit systems sometimes use unverifiable assertions. This is a classification error, and it has a precise form: an assertion of type `Undefined` is used where an assertion of type `Verified` is required. This paper develops a discipline that makes the error structurally inadmissible. The discipline rests on one invariant: no audit conclusion may depend on a material assertion whose verification state is undefined. The paper defines a typed state space for audit assertions, an admissibility rule restricting unqualified conclusions to admitted `Verified` assertions, preconditions and postconditions for verification procedures, a residual register that preserves everything a procedure does not establish, and a composition rule under which an opinion inherits the weakest unresolved material assertion on which it depends. `Undefined` is presented throughout as a success state: it is the correct output of a classifier that has refused to speak beyond its boundary. The Wirecard collapse is treated not as motivation but as a classification trace exhibiting a violation of the invariant, and continuous auditing exception queues and SQL's three-valued `Unknown` supply the same violation at ordinary scale. A machine-checked sketch of the central admissibility theorem demonstrates that the discipline is implementable. The method does not prevent fraud. It prevents an audit classification system from treating an undefined material assertion as verified, which is a narrower and therefore provable claim.

Keywords audit classification · verification failure · partial functions · admissibility · three-valued logic · formal methods · accounting information systems

1 The Problem

An audit is, among other things, a classification system. It receives assertions together with evidence and it produces classifications on which conclusions depend. The problem addressed in this paper is that audit systems sometimes use unverifiable assertions, and that they do so silently. An assertion that could not be verified

D. Moore
Independent Researcher, Edmonton, Alberta, Canada
E-mail: dhwcmoore@gmail.com

within the evidence, authority, and observation boundaries available to the system is nonetheless treated, downstream, as though it had been verified.

This is not a moral failure and it is not, in the first instance, a regulatory failure. It is a type error. An assertion of type `Undefined` is being used where an assertion of type `Verified` is required. The remainder of this paper is the discipline that makes the type error inadmissible.

Three terms are needed and are used consistently throughout. *Verification failure* is the condition in which a system cannot determine whether a claim is verified or refuted within its available boundaries. An *implicit default* is the local rule, convention, or implementation path that fills the resulting gap, typically with acceptance. *Silent conversion* is the system-level result: the unresolved claim is treated as accepted, and no explicit failure state records that this happened.

The classification function of a conventional audit system is total. It produces an output for every input, including inputs over which it has no verified authority, and the convenient output for such inputs is acceptance. The classification function of a disciplined audit system is partial, and honestly so. Where it has no authority, it says so:

<code>cannot verify -> accept</code>	<code>(undisciplined)</code>
<code>cannot verify -> Undefined</code>	<code>(disciplined)</code>

The whole paper is contained in the difference between those two lines. `Undefined` is not a defect of the disciplined classifier. It is the sign that the classifier has refused to lie.

One rule governs everything that follows.

Rule 1.1 *Do not say that the system provides assurance. Say exactly what proposition it verifies.*

The central invariant can now be stated. Everything else in the paper is support for it.

Rule 1.2 (Central Invariant) *No audit conclusion may depend on a material assertion whose verification state is undefined.*

In symbolic form:

<code>Admissible(Opinion) =></code> <code> for every material assertion a used by Opinion:</code> <code> state(a) = Verified</code>
--

Equivalently, and more sharply:

<code>state(a) = Undefined => a is inadmissible for an unqualified opinion</code>
--

The paper proceeds by construction. Section 2 defines the state space. Section 3 defines admissibility. Section 4 defines boundaries and gives every verification procedure a precondition and a postcondition. Section 5 defines residuals and the register that preserves them. Section 6 gives the composition rule for opinions. Section 7 exhibits three violations of the invariant: Wirecard as a classification trace, continuous auditing exception queues as an algorithm, and SQL `Unknown` as a semantic error at the application layer. Section 8 gives the formal sketch.

Section 9 states, briefly, what existing literatures supply and what they do not. Section 10 draws consequences for accounting information systems, for standards, and for AI-assisted audit tools.

2 States

Loose words such as assurance, reasonable, sufficient, and appropriate are admitted into this paper only after they have been turned into typed states. The state space is declared once and used everywhere.

```

AuditAssertion ::=
  claim about existence
  | claim about valuation
  | claim about completeness
  | claim about rights and obligations
  | claim about control

EvidenceState ::=
  Verified
  | Refuted
  | Undefined
  | PresumptivelyVerified
  | PresumptivelyRefuted
  | EscalationRequired

AuditUse ::=
  Admissible
  | Inadmissible

```

The BNF above fixes the vocabulary informally; the Coq/Rocq development declares the same states as an inductive type, so that the formal sketch of Section 8 is stated over exactly this vocabulary rather than a re-encoding of it:

```

Inductive EvidenceState : Type :=
  | Verified
  | Refuted
  | Undefined
  | PresumptivelyVerified
  | PresumptivelyRefuted
  | EscalationRequired.

Inductive AuditUse : Type :=
  | Admissible
  | Inadmissible.

```

The states have the following meanings and no others. **Verified**: the assertion falls within a declared boundary and the evidence grade that boundary requires is present. **Refuted**: admissible evidence establishes that the assertion is false. **Undefined**: the assertion falls outside all declared boundaries, or the required evidence grade is absent, and no admissible procedure currently resolves it. **PresumptivelyVerified** and **PresumptivelyRefuted**: available evidence leans one way without meeting the declared grade for the assertion's materiality. **EscalationRequired**: the assertion exposes a defect in the boundary specification itself, such as a conflict, an overlap, or an assertion class for which no boundary was declared.

Remark 2.1 Undefined is a success state. A classifier that returns Undefined for an input outside its verified domain has computed the correct answer. The failure states of an audit classification system are not its Undefined outputs; they are its affirmative outputs on inputs it had no authority to classify.

3 Admissibility

Definition 3.1 (Admissibility) A material assertion a is admissible for an unqualified audit conclusion if and only if $\text{state}(a) = \text{Verified}$ under the declared boundary specification, and the classification was admitted by a verifier independent of the process that proposed it.

```
admissible(a) = true
  iff EvidenceState(a) = Verified
  and admitted_by(independent_verifier, a)
```

Both conjuncts are load-bearing. The first excludes every non-Verified state; the second excludes a Verified classification that a proposing process assigned to its own output, which is precisely the failure mode Theorem 8.3 rules out.

Presumptive states may guide triage. They order queues, allocate investigative attention, and inform planning. They may not support an unqualified conclusion. The moment a presumptive state binds consequence, the discipline has been abandoned, because a presumptive state is precisely a state whose evidence does not meet the declared grade.

Professional judgement is not denied by this rule; it is relocated. Judgement enters only in the declaration of thresholds, evidence classes, boundaries, and admissibility rules. Once declared, those rules are applied mechanically. Judgement is exercised at specification time. It is not available as an invisible runtime patch applied to individual assertions after the evidence has disappointed.

4 Boundaries and Preconditions

Definition 4.1 (Boundary) A boundary is the declared region, in assertion class, jurisdiction, evidence grade, and materiality, inside which a verification procedure produces valid results. A verification procedure is valid only inside its declared boundary.

Every verification procedure carries a proof obligation of the same shape as a program statement: a precondition stating what must be true before the procedure is entitled to produce a verification result, and a postcondition stating exactly which classification the procedure is entitled to produce. The question to ask of every audit procedure is the weakest-precondition question: what must hold for this procedure's output to mean what it claims to mean?

```
Procedure: Direct bank confirmation

Preconditions:
  bank exists as independent counterparty
  confirmation route is controlled by the auditor
```

```

jurisdiction permits direct confirmation
account identifier is supplied
response is received through an authenticated channel

```

Postcondition:
the cash-existence assertion may be classified Verified

If any precondition fails, the procedure cannot produce Verified. Not should not: cannot, within the declared system. A confirmation received through a route the auditor does not control, or from a jurisdiction that obstructs direct confirmation, does not weakly satisfy the precondition; it fails it, and the postcondition is unavailable. Silent conversion, in this vocabulary, is simply a postcondition violation: a procedure whose preconditions failed was nonetheless treated as having discharged its postcondition.

Boundaries are declared as versioned, machine-readable data, so that the precondition check is itself mechanical. The declaration enumerates, for each procedure, the assertion classes it covers, the jurisdictions in which its preconditions are satisfiable, the evidence grade it produces, and the materiality that grade can support. An assertion for which no declared procedure's preconditions hold is classified Undefined by construction.

5 Residuals and the Verification Balance Sheet

Anything outside the boundary is not erased. It is registered.

Definition 5.1 (Verification Balance Sheet) For a verification procedure V :

```

VerificationBalanceSheet(V) =
  Assumptions(V)    what V takes as given without verification
  Target(V)         exactly what successful completion establishes
  Cost(V)           what completion consumes
  Residuals(V)      what remains unestablished after completion
  TrustChain(V)     the delegations connecting V to its assumptions

```

subject to the constraint:

```

Target(V) must not include any claim in Residuals(V)

```

The constraint is the point. The verified target must not smuggle in its own residuals. A procedure whose stated target quietly includes claims that its own residual list disclaims has misstated what it verifies, and every downstream use of its output inherits the misstatement. Rule 1.1 is enforced here: the Target line is the exact proposition the procedure verifies, nothing more, and the Residuals line enumerates what a reader might wrongly assume is covered.

Definition 5.2 (Residual Register) Every classification other than an admitted Verified writes an entry to a residual register recording the assertion, the state assigned, the boundary evaluation that produced it, and an identified owner for the residual risk or an explicit orphan flag triggering escalation. The register is append-only with respect to workflow: no queue-clearing, phase-completion, or dashboard operation deletes or downgrades an entry.

Durability is what distinguishes a residual register from an exception log. An exception log records that something was flagged. A residual register records that something remains unestablished, and it continues to record this after the workflow that flagged it has closed.

6 Composition

Audits are made of many procedures, and an opinion depends on many assertions. The discipline is worthless unless correct local classifications compose into a correct conclusion, so the composition rule is stated explicitly.

Rule 6.1 (Composition) *An audit opinion is admissible as unqualified only if every material assertion on which it depends is admissible. An opinion inherits the weakest unresolved material assertion on which it depends.*

```
if exists a in deps(Opinion) such that not admissible(a)
then Opinion is inadmissible as unqualified

(* admissible(a) is false whenever
   state(a) in {Undefined, Refuted, PresumptivelyVerified,
                PresumptivelyRefuted, EscalationRequired}
   or state(a) = Verified but admission was not independent *)
```

The inheritance direction matters. Strength does not propagate upward from the majority of well-verified assertions; weakness propagates upward from the single unresolved one. This is not pessimism. It is the observation that an unqualified opinion asserts a conjunction, and a conjunction is no stronger than its weakest conjunct. A methodology that averages evidence quality across assertions, or that permits a preponderance of Verified classifications to carry an Undefined one across the admissibility line, has reintroduced the implicit default at the composition layer after eliminating it at the classification layer.

7 Three Violations of the Invariant

The invariant was stated before any example, and the discipline stands whether or not the following cases had occurred. The cases are exhibited because each violates the invariant in a different layer: the first in engagement-level classification, the second in workflow, the third in application semantics.

7.1 Wirecard as a Classification Trace

The collapse of Wirecard AG in 2020 is commonly narrated as a failure of scepticism or an achievement of fraud (McCrum 2022; ESMA 2020). Under the present discipline it is neither. It is a misclassification, and it can be exhibited as a trace.

```
Assertion:
A = "Wirecard holds EUR 1.9bn in Philippine trustee accounts."
```

```

Required boundary for material cash existence:
  direct bank confirmation, authenticated route,
  cooperative jurisdiction

Observed evidence:
  no authenticated direct bank confirmation
  intermediary documentation (screenshots, trustee statements)
  jurisdictional obstruction

Precondition check (direct bank confirmation):
  confirmation route controlled by auditor .... FAILED
  jurisdiction permits direct confirmation ... FAILED

Classification under the discipline:
  state(A) = Undefined

Admissibility:
  A inadmissible for an unqualified opinion    (Rule 1.2, Central Invariant)

Legacy outcome:
  A treated as accepted; unqualified opinions issued

Violation:
  Undefined(A) was converted into Accepted(A)

```

The claim made by this trace is deliberately narrow. The discipline would not have prevented the fraud, and it compels no particular exercise of judgement. It would have changed the classification state of assertion *A*: the evidential obstruction would have been recorded as *Undefined*, with a registered residual and an inadmissible unqualified opinion, rather than passing as ordinary acceptance. The jurisdictional, evidence-quality, and control-boundary failures that enabled the fraud are, in this vocabulary, three failed preconditions whose failure the legacy system did not record.

7.2 Continuous Auditing Queues as an Algorithm

The same violation occurs at ordinary scale in continuous auditing, and there it can be written as a three-line algorithm. Suppose a monitoring system flags 10,000 transactions in a period and a thresholding layer routes 200 to human review. The undisciplined system runs:

```

for each exception e:
  if priority(e) < threshold:
    clear(e)

```

The 9,800 cleared exceptions are not verified; they are deprioritised. If the dashboard then reports that 200 exceptions were reviewed with no material issue found, while the 9,800 vanish from consequence, the system has silently converted unreviewed exceptions into accepted background. The disciplined system runs:

```

for each exception e:
  if priority(e) < threshold:
    state(e) := Unreviewed
    residual_register.add(e)

```

with the governing inequality:

Unreviewed != Accepted

That single line is the continuous auditing case in full. Prioritisation is legitimate; capacity is finite; triage is not the error. The error is the disappearance of the deprioritised exceptions from the record of what remains unestablished. Under the discipline they persist in the residual register, and the aggregate of such entries is itself evidence about the assurance the monitoring system actually provides, as distinct from the assurance its cleared dashboard displays (Alles et al. 2006; Issa and Kogan 2014; Austin et al. 2021).

7.3 SQL Unknown as a Semantic Error

At the data layer, accounting information systems already possess the required third value. SQL evaluates comparisons involving missing data to Unknown rather than to true or false (Codd 1979). The database refuses to convert absence of information into an affirmative truth value.

The error occurs above the database, and it is a semantic error with a precise description. The error is not that the database has an Unknown value. The error is that the surrounding system lacks the discipline to preserve it. A reporting layer that renders a null risk score as zero, a middleware coercion that aggregates a missing confirmation as no exception, and a filter predicate that silently excludes unresolved rows so that a completeness check passes are all instances of the same conversion:

Unknown at the data layer must not become Accepted at the audit layer

The database example demonstrates that the technical capacity to represent verification failure has existed for decades. What has been missing is not the third value but the discipline that preserves it through every layer standing between the evidence and the conclusion.

8 Formal Sketch

The formalisation target is deliberately narrow and structural. The proof assistant is not verifying the audit. It is verifying the classification rule that prevents an unverifiable material assertion from being admitted as verified. Substantive testing, sampling sufficiency, valuation, and fraud detection are not formalised; the discipline constrains how their results are allowed to propagate into conclusions.

Theorem 8.1 (Default-Free Admissibility) *For any audit opinion O , if O is unqualified, then every material assertion on which O depends is classified **Verified** under the declared boundary specification and admitted by an independent verifier.*

Corollary 8.1 (Contrapositive) *If any material assertion on which O depends is **Undefined**, then O is not admissible as an unqualified opinion.*

Theorem 8.2 (Residual Preservation) *Every non-affirmative classification of a material assertion writes an entry to the residual register, and no workflow operation deletes or downgrades an entry.*

Theorem 8.3 (No Self-Admission) *A proposed classification cannot admit itself. Admission is performed only by a verifier independent of the process that generated the proposal.*

The main theorem is stated in Coq as follows; the development extracts to an executable OCaml classifier that satisfies the theorems by construction.

```
Theorem default_free_admissibility :
  forall (O : Opinion),
    unqualified O ->
      forall (a : Assertion),
        In a (deps O) -> material a ->
          state a = Verified /\ independently_admitted a.
```

The predicate is named `independently_admitted` rather than `admitted` precisely because `Admitted` is a reserved proof-closing command in Coq/Rocq; naming a semantic predicate identically to that tactic is the kind of small inconsistency the discipline itself warns against, so the sketch avoids it throughout. The `state` function is not primitive. Its value is computed from the declared boundary specification by the precondition checks of Section 4, so that verifiability is determined by declared boundaries and mechanical application, not by discretion at the point of disappointment. The force of the theorem lies in what it excludes: within the declared system there exists no derivation of an unqualified opinion from a dependency set containing an Undefined material assertion. The type error of Section 1 is not discouraged. It is unrepresentable.

The remaining two theorems admit the same treatment. Residual preservation is a monotonicity property on the register viewed as a growing log, and no-self-admission is a separation property between the process that proposes a classification and the process that admits it:

```
Theorem residual_preservation :
  forall (log1 log2 : RegisterLog) (e : ResidualEntry),
    step log1 log2 -> In e log1 -> In e log2.

Theorem no_self_admission :
  forall (p : Process) (a : Assertion),
    proposes p a -> ~ admits p a.
```

`step` is the register’s transition relation under any workflow operation (queue-clearing, phase completion, dashboard refresh); the theorem states that every such step is log-extending, never log-shrinking. `admits` is deliberately a relation on the process performing admission rather than on the assertion alone, so that `no_self_admission` cannot be discharged by a proposing process simply relabelling itself.

Remark 8.1 (On the word “machine-checked”) The abstract’s phrase “machine-checked sketch” hedges the scope of what is formalised, not the generality of what is proved. `default_free_admissibility`, `residual_preservation`, and `no_self_admission` are proved in the accompanying development as stated above: as theorems quantified

over abstract, unbounded `Assertion/Opinion/Process` types, not restricted to the finite instances exercised there, and `coqchk` confirms every module in the development is closed under the global context with no `Admitted` and no `Axiom`. The three worked cases (Wirecard, a continuous-auditing exception queue, and the SQL Unknown collapse) exercise these general theorems on finite instances; they illustrate the theorems and are not the extent of what is proved. What remains a sketch is the object of formalisation, not its rigor: the development verifies the classification rule of this section, not an audit, and the extraction to the executable OCaml classifier is, as with any Coq extraction, trusted rather than independently machine-checked.

Artifact availability

In keeping with this journal’s emphasis on tool and proof replicability, the Coq/Rocq development and the OCaml classifier it extracts to are intended to accompany this submission as an online appendix or linked repository at the final-version stage; *[repository URL to be inserted before submission]*. The evaluation results reported in Section 7 are illustrative traces rather than outputs of a run over a production dataset, and the repository README states this distinction explicitly so that reviewers do not mistake the Wirecard and continuous-auditing examples for empirical findings.

Note on a related submission

A companion manuscript, *Boundary Discipline for Accounting Information Systems: On the Correct Treatment of Undefined Audit Assertions*, has been submitted to the *International Journal of Digital Accounting Research*. Both manuscripts develop the same central invariant (Rule 1.2) and share the Coq/Rocq development described above. The present manuscript addresses this journal’s formal-methods and tooling audience, and takes the mechanised admissibility theorem and its Coq/Rocq development as its central technical contribution. The companion manuscript instead addresses an accounting information systems audience, and develops the surrounding architecture, the Verification Balance Sheet, the Risk Assignment Register, and standards and inspection implications that are outside the scope of the present submission. We disclose the relationship so that editors and reviewers can evaluate the degree of overlap directly; in our own assessment the two manuscripts share an invariant and a proof artefact but not a contribution, and neither manuscript’s findings depend on prior acceptance of the other.

9 What Existing Literatures Supply

The related literatures each recognise part of the problem, and the purpose of this section is to establish exactly one claim: none of them supplies an audit-specific discipline preventing `Undefined` from being used as `Verified`. The typed state space of Section 2 is, in the vocabulary of this journal’s own abstract-interpretation literature, a coarse abstraction of a much richer evidential domain (Rustenholz et

al. 2026); the present paper does not develop that lattice-theoretic apparatus, and confines itself to the six-state abstraction of Section 2 because that is the granularity at which the admissibility question of Section 3 is decidable by inspection.

Audit theory distinguishes what an auditor establishes from what an auditor merely fails to refute, and treats evidential sufficiency as the determinant of supportable assurance (Mautz and Sharaf 1961; Knechel et al. 2013; Nelson 2009; Hurtt 2010). Its standards provide an output channel for verification failure in the modified opinion (IAASB 2016), and the empirical literature shows the channel operating under judgement and incentive rather than under any classification architecture (Habib 2013). The recognition is real; the enforcement is behavioural, and nothing in the classification logic makes the conversion of an unresolved assertion into an accepted one unrepresentable.

Accounting information systems research supplies continuous monitoring, exception processing, and analytics (Vasarhelyi and Halper 1991; Kogan et al. 1999; Alles et al. 2006; Austin et al. 2021), and it documents the workflow conditions under which deprioritised exceptions are operationally absorbed (Issa and Kogan 2014). It lacks a state in which an unreviewed exception durably remains unreviewed after the dashboard has been cleared.

Database theory supplies the third value itself (Codd 1979) and preserves it at the data layer, but permits every layer above to destroy it. Runtime verification supplies multi-valued verdicts over incomplete observations, with inconclusive as a first-class outcome and presumptive refinements that guide without binding (Bauer et al. 2011, 2010; Leucker and Schallhart 2009), but this machinery has not been applied to audit evidence. Recent work on quantitative robustness for temporal specifications continues this line by replacing a binary verdict with a graded distance to the satisfaction boundary (Rino et al. 2026); that measure is a refinement of the *qualitative* verdict, useful for triage in exactly the sense Section 3 grants to presumptive states, and it is not itself an admissibility criterion, which is the distinction this paper insists on.

Each literature holds a piece. The discipline of this paper is the piece none of them holds: a typed classification of audit assertions in which `Undefined` is preserved from the point of verification failure through composition to audit consequence, and in which the admissibility of the conclusion is machine-checkable.

10 Implications

For accounting information systems, the discipline is an architecture: claim packets carrying assertions and evidence, boundary specifications as versioned data, a classification function returning the typed states of Section 2, an admission verifier separated from every proposing process, and an append-only residual register. The architecture is fail-closed: an assertion for which no declared procedure’s preconditions hold cannot reach an affirmative state by any path.

For standards and inspection, the discipline yields measurable indicators rather than declared policy: the fraction of material assertions falling within declared boundaries, the materiality resting in regions whose correct classification is `Undefined`, and the fraction of residuals with an identified owner. A standard could require the declarations and the register; an inspection could measure the indicators; illustrative requirement language is confined to Appendix A, because standards are

administration and the contribution of this paper is the discipline. Once residuals are explicit they can also be priced, insured, or disclosed, but pricing is a consequence of the discipline, not a component of it.

For AI-assisted audit tools, the discipline becomes more urgent without becoming different. A generated conclusion is a proposed classification. Under Theorem 8.3 it cannot admit itself, and under Rule 1.2 it cannot bind consequence until an independent verifier classifies it Verified within a declared boundary. This is not a hypothetical caution: a recent benchmark of large language models on semantic-equivalence judgments over short code samples found substantial misclassification rates even with supporting context (Laneve et al. 2026), which is exactly the failure mode Theorem 8.3 is positioned against if a model’s own output were permitted to double as its admission. The treatment of generation as proposal requiring admission is the entire extension, and its development belongs to separate work.

11 Conclusion

No method prevents fraud. The object of this method is not to make fraud impossible. It is to make the acceptance of unverifiable assertions impossible within the declared classification system. The audit classifier of this paper cannot honestly record an unverifiable material assertion as verified: the state space contains Undefined, the boundary specification computes it, the residual register preserves it, the composition rule propagates it, and the admissibility theorem excludes any unqualified conclusion that depends on it. What has been verified is exactly what the balance sheet’s Target line states, nothing more, and what has not been verified is registered rather than erased. An audit system built to this discipline is humbler than its predecessors in precisely one respect: it knows when it is not entitled to speak.

References

- Alles, M. G., Brennan, G., Kogan, A., and Vasarhelyi, M. A. (2006). Continuous monitoring of business process controls: A pilot implementation of a continuous auditing system at Siemens. *International Journal of Accounting Information Systems*, 7(2), 137–161. <https://doi.org/10.1016/j.accinf.2005.10.004>
- Austin, A. A., Carpenter, T. D., Christ, M. H., and Nielson, C. S. (2021). The data analytics journey: Interactions among auditors, managers, regulation, and technology. *Contemporary Accounting Research*, 38(3), 1888–1924. <https://doi.org/10.1111/1911-3846.12680>
- Bauer, A., Leucker, M., and Schallhart, C. (2010). Comparing LTL semantics for runtime verification. *Journal of Logic and Computation*, 20(3), 651–674. <https://doi.org/10.1093/logcom/exn075>
- Bauer, A., Leucker, M., and Schallhart, C. (2011). Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4), Article 14. <https://doi.org/10.1145/2000799.2000800>
- Codd, E. F. (1979). Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4), 397–434. <https://doi.org/10.1145/320107.320109>
- European Securities and Markets Authority (2020). *Fast Track Peer Review on the Application of the Guidelines on the Enforcement of Financial Information by BaFin and FREP in the Context of Wirecard*. ESMA42-111-5349. Paris: ESMA. No DOI assigned.
- Habib, A. (2013). A meta-analysis of the determinants of modified audit opinion decisions. *Managerial Auditing Journal*, 28(3), 184–216. <https://doi.org/10.1108/02686901311304349>

- Hurt, R. K. (2010). Development of a scale to measure professional skepticism. *Auditing: A Journal of Practice & Theory*, 29(1), 149–171. <https://doi.org/10.2308/aud.2010.29.1.149>
- International Auditing and Assurance Standards Board (2016). *International Standard on Auditing 705 (Revised): Modifications to the Opinion in the Independent Auditor’s Report*. New York: IFAC. No DOI assigned.
- Issa, H. and Kogan, A. (2014). A predictive ordered logistic regression model as a tool for quality review of control risk assessments. *Journal of Information Systems*, 28(2), 209–229. <https://doi.org/10.2308/isys-50808>
- Knechel, W. R., Krishnan, G. V., Pevzner, M., Shefchik, L. B., and Velury, U. K. (2013). Audit quality: Insights from the academic literature. *Auditing: A Journal of Practice & Theory*, 32(Supplement 1), 385–421. <https://doi.org/10.2308/ajpt-50350>
- Kogan, A., Sudit, E. F., and Vasarhelyi, M. A. (1999). Continuous online auditing: A program of research. *Journal of Information Systems*, 13(2), 87–103. <https://doi.org/10.2308/jis.1999.13.2.87>
- Laneve, C., Spanò, A., Ressi, D., et al. (2026). Understanding code semantics: a benchmark study of LLMs. *International Journal on Software Tools for Technology Transfer*, 28, 329–343. <https://doi.org/10.1007/s10009-026-00842-4>
- Leucker, M. and Schallhart, C. (2009). A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5), 293–303. <https://doi.org/10.1016/j.jlap.2008.08.004>
- Mautz, R. K. and Sharaf, H. A. (1961). *The Philosophy of Auditing*. Sarasota, FL: American Accounting Association.
- McCrum, D. (2022). *Money Men: A Hot Startup, a Billion Dollar Fraud, a Fight for the Truth*. London: Bantam Press.
- Nelson, M. W. (2009). A model and literature review of professional skepticism in auditing. *Auditing: A Journal of Practice & Theory*, 28(2), 1–34. <https://doi.org/10.2308/aud.2009.28.2.1>
- Rino, N., Foughali, M., Renkin, F., and Asarin, E. (2026). Efficiently computable temporal robustness for a practical STL fragment. *International Journal on Software Tools for Technology Transfer*, 28, 71–103. <https://doi.org/10.1007/s10009-026-00852-2>
- Rustenholtz, L., Lopez-Garcia, P., and Hermenegildo, M. V. (2026). Abstractions of sequences, functions and operators. *International Journal on Software Tools for Technology Transfer*, 28, 277–301. <https://doi.org/10.1007/s10009-026-00843-3>
- Vasarhelyi, M. A. and Halper, F. B. (1991). The continuous audit of online systems. *Auditing: A Journal of Practice & Theory*, 10(1), 110–125. No DOI assigned.

A Illustrative Requirement Language

The following requirements illustrate how the discipline could be codified in a standard, an inspection protocol, or an internal methodology. They are illustrative rather than proposed regulatory text.

1. Verification boundaries must be declared for all material assertion classes as versioned, machine-readable data specifying procedures, jurisdictions, evidence grades, and supportable materiality, together with the precondition set of each procedure.
2. Classification of assertions against declared boundaries must be mechanical, and assertions for which no declared procedure’s preconditions hold must be classified **Undefined**.
3. Every non-affirmative classification of a material assertion must write a durable entry to a residual register identifying the residual and its owner, with orphaned residuals escalated.
4. An unqualified conclusion is admissible only if every material assertion on which it depends is an admitted **Verified** classification, where admission is performed by a verifier independent of every proposing process.
5. Inspection must measure boundary coverage, undefined-region materiality, and residual assignment from the declared specifications and classification logs rather than from declared policy.